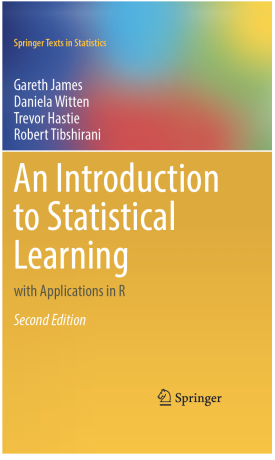
ML101
Chapter 10: Neural Networks
An Introduction to Statistical Learning, ISLRv2

James, Witten, Hastie & Tibshirani

November 5, 2026

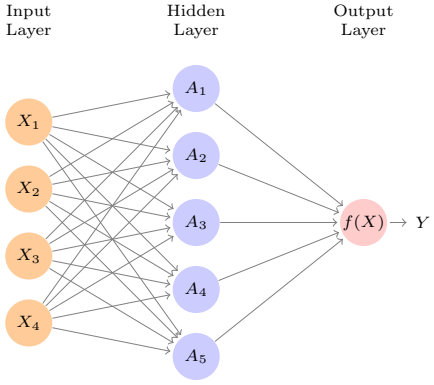
An Introduction to Statistical Learning with Applications in R

a.k.a. ISLR



Content has been extracted from *An Introduction to Statistical Learning with Applications in R*, Second Edition, by James, Witten, Hastie & Tibshirani. Springer. 2023.
Visit https://www.statlearning.com/.

Fitting Neural Networks

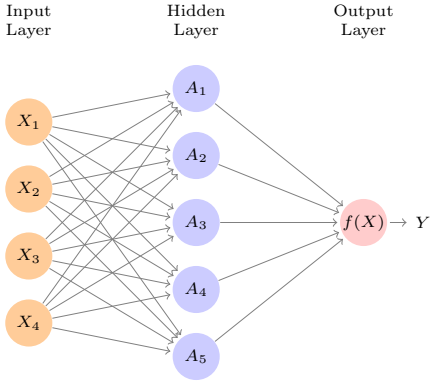


$$\min_{\{w_k\}_1^K,\beta} \frac{1}{2}\sum_{i=1}^{n}(y_i - f(x_i))^2,$$

where

$$f(x_i) = \beta_0 + \sum_{k=1}^{K} \beta_k g\left(w_{k0} + \sum_{j=1}^{p} w_{kj}x_{ij}\right).$$

Fitting Neural Networks



$$\min_{\{w_k\}_1^K,\beta} \frac{1}{2}\sum_{i=1}^{n}(y_i - f(x_i))^2,$$

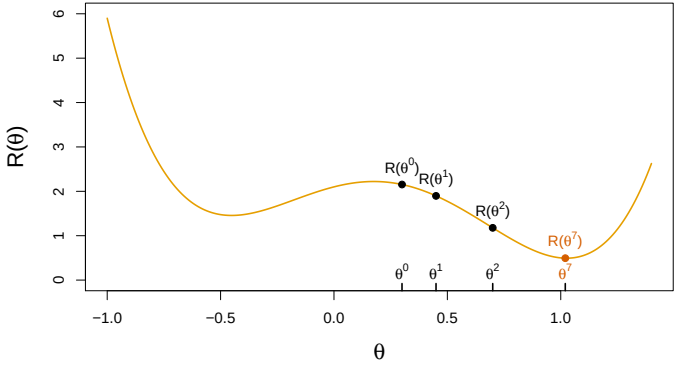
where

$$f(x_i) = \beta_0 + \sum_{k=1}^{K} \beta_k g\left(w_{k0} + \sum_{j=1}^{p} w_{kj}x_{ij}\right).$$

This problem is difficult because the objective is **non-convex**. Despite this, effective algorithms have evolved that can optimize complex neural network problems efficiently.

Non Convex Functions and Gradient Descent

Let $R(\theta) = \frac{1}{2}\sum_{i=1}^{n}(y_i - f_\theta(x_i))^2$ with $\theta = (\{w_k\}_1^K, \beta)$.



1. Start with a guess θ^0 for all the parameters in θ, and set $t = 0$.
2. Iterate until the objective $R(\theta)$ fails to decrease:
   - 2.1 Find a vector δ that reflects a small change in θ, such that $\theta^{t+1} = \theta^t + \delta$ **reduces** the objective; i.e. $R(\theta^{t+1}) < R(\theta^t)$.
   - 2.2 Set $t \leftarrow t + 1$.

Gradient Descent Continued

- In this simple example we reached **the global minimum**.
- If we had started a little to the left of θ^0 we would have gone in the other direction, and ended up in a **local minimum**.
- Although θ is multi-dimensional, we have depicted the process as one-dimensional. It is much harder to identify whether one is in a local minimum in high dimensions.

How to find a direction δ that points downhill? We compute the **gradient vector**

$$\nabla R(\theta^t) = \left.\frac{\partial R(\theta)}{\partial \theta}\right|_{\theta=\theta^t}$$

i.e. the vector of **partial derivatives** at the current guess θ^t.

The gradient points uphill, so our update is $\delta = -\rho \nabla R(\theta^t)$ or

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R(\theta^t)$$

where ρ is the **learning rate** (typically small, e.g. $\rho = 0.001$).

Gradients and Backpropagation

$R(\theta) = \sum_{i=1}^{n} R_i(\theta)$ is a sum, so gradient is sum of gradients.

$$R_i(\theta) = \frac{1}{2}(y_i - f_\theta(x_i))^2 = \frac{1}{2}\left(y_i - \beta_0 - \sum_{k=1}^{K} \beta_k g\left(w_{k0} + \sum_{j=1}^{p} w_{kj} x_{ij}\right)\right)^2$$

For ease of notation, let $z_{ik} = w_{k0} + \sum_{j=1}^{p} w_{kj} x_{ij}$.

Backpropagation uses the **chain rule for differentiation**:

$$\begin{aligned}\frac{\partial R_i(\theta)}{\partial \beta_k} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial \beta_k} \\ &= -(y_i - f_\theta(x_i)) \cdot g(z_{ik}).\end{aligned}$$

$$\begin{aligned}\frac{\partial R_i(\theta)}{\partial w_{kj}} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial g(z_{ik})} \cdot \frac{\partial g(z_{ik})}{\partial z_{ik}} \cdot \frac{\partial z_{ik}}{\partial w_{kj}} \\ &= -(y_i - f_\theta(x_i)) \cdot \beta_k \cdot g'(z_{ik}) \cdot x_{ij}.\end{aligned}$$

Tricks of the Trade

- **Slow learning**. Gradient descent is slow, and a small learning rate ρ slows it even further. With early stopping, this is a form of regularization.

Tricks of the Trade

- **Slow learning**. Gradient descent is slow, and a small learning rate ρ slows it even further. With early stopping, this is a form of regularization.
- **Stochastic gradient descent**. Rather than compute the gradient using all the data, use a small minibatch drawn at random at each step. E.g. for MNIST data, with $n = 60K$, we use minibatches of 128 observations.

Tricks of the Trade

- **Slow learning**. Gradient descent is slow, and a small learning rate ρ slows it even further. With early stopping, this is a form of regularization.
- **Stochastic gradient descent**. Rather than compute the gradient using all the data, use a small minibatch drawn at random at each step. E.g. for MNIST data, with $n = 60K$, we use minibatches of 128 observations.
- An **epoch** is a count of iterations and amounts to the number of minibatch updates such that n samples in total have been processed; i.e. $60K/128 \approx 469$ for MNIST.

Tricks of the Trade

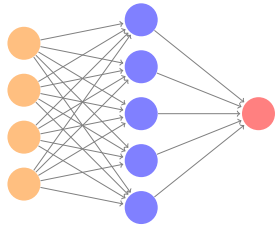
- **Slow learning**. Gradient descent is slow, and a small learning rate ρ slows it even further. With early stopping, this is a form of regularization.
- **Stochastic gradient descent**. Rather than compute the gradient using all the data, use a small minibatch drawn at random at each step. E.g. for MNIST data, with $n = 60K$, we use minibatches of 128 observations.
- An **epoch** is a count of iterations and amounts to the number of minibatch updates such that n samples in total have been processed; i.e. $60K/128 \approx 469$ for MNIST.
- **Regularization**. Ridge and lasso regularization can be used to shrink the weights at each layer. Two other popular forms of regularization are dropout and augmentation, discussed next.

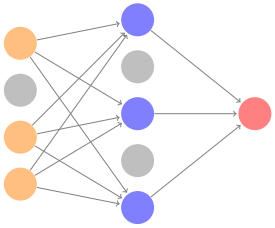
Dropout Learning



- At each SGD update, randomly remove units with probability ϕ, and scale up the weights of those retained by $1/(1-\phi)$ to compensate.

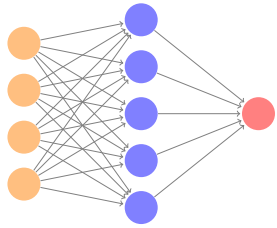
Dropout Learning

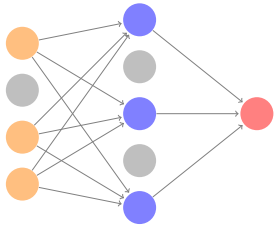




- At each SGD update, randomly remove units with probability ϕ, and scale up the weights of those retained by $1/(1-\phi)$ to compensate.
- In simple scenarios like linear regression, a version of this process can be shown to be equivalent to ridge regularization.

Dropout Learning





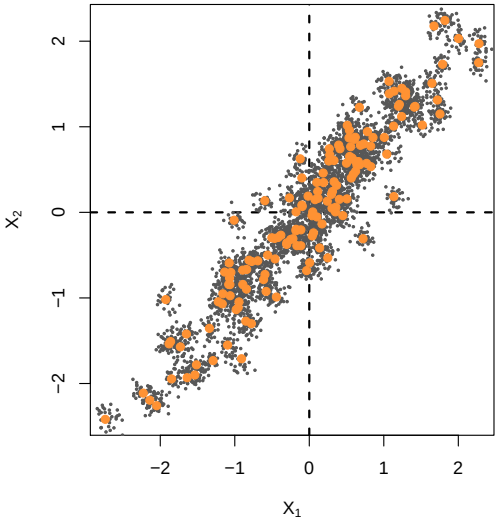
- At each SGD update, randomly remove units with probability ϕ, and scale up the weights of those retained by $1/(1-\phi)$ to compensate.
- In simple scenarios like linear regression, a version of this process can be shown to be equivalent to ridge regularization.
- As in ridge, the other units **stand in** for those temporarily removed, and their weights are drawn closer together.

Dropout Learning



- At each SGD update, randomly remove units with probability ϕ, and scale up the weights of those retained by $1/(1-\phi)$ to compensate.
- In simple scenarios like linear regression, a version of this process can be shown to be equivalent to ridge regularization.
- As in ridge, the other units **stand in** for those temporarily removed, and their weights are drawn closer together.
- Similar to randomly omitting variables when growing trees in random forests (Chapter 8).

Ridge and Data Augmentation



- Make many copies of each (x_i, y_i) and add a small amount of Gaussian noise to the x_i – a little cloud around each observation – but leave the copies of y alone!
- This makes the fit robust to small perturbations in x_i, and is equivalent to ridge regularization in an OLS setting.

Data Augmentation on the Fly



- Data augmentation is especially effective with SGD, here demonstrated for a CNN and image classification.

Data Augmentation on the Fly



- Data augmentation is especially effective with SGD, here demonstrated for a CNN and image classification.
- Natural transformations are made of each training image when it is sampled by SGD, thus ultimately making a cloud of images around each original training image.

Data Augmentation on the Fly



- Data augmentation is especially effective with SGD, here demonstrated for a CNN and image classification.
- Natural transformations are made of each training image when it is sampled by SGD, thus ultimately making a cloud of images around each original training image.
- The label is left unchanged in each case – still tiger.

Data Augmentation on the Fly



- Data augmentation is especially effective with SGD, here demonstrated for a CNN and image classification.
- Natural transformations are made of each training image when it is sampled by SGD, thus ultimately making a cloud of images around each original training image.
- The label is left unchanged in each case – still tiger.
- Improves performance of CNN and is similar to ridge.

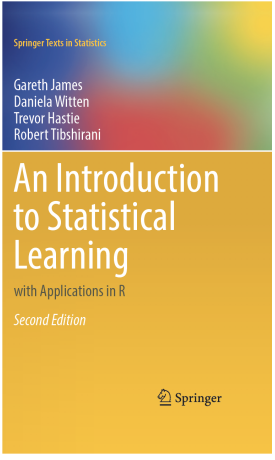
Software

- Wonderful software available for neural networks and deep learning. `Tensorflow` from Google and `PyTorch` from Facebook. Both are `Python` packages.
- In the Chapter 10 lab we demonstrate `tensorflow` and `keras` packages in `R`, which interface to `Python`. See textbook and online resources for `Rmarkdown` and `Jupyter` notebooks for these and all labs for the second edition of ISLR book.
- The `torch` package in `R` is available as well, and implements the `PyTorch` dialect. The Chapter 10 lab will be available in this dialect as well; watch the resources page at `www.statlearning.com`.

End of Chapter 10.

An Introduction to Statistical Learning with Applications in R

a.k.a. ISLR



Content has been extracted from *An Introduction to Statistical Learning with Applications in R*, Second Edition, by James, Witten, Hastie & Tibshirani. Springer. 2023.
Visit https://www.statlearning.com/.